

# Preprocessing and clustering 3k PBMCs (legacy workflow)

Print to PDF

## Contents

- Preprocessing
- Principal component analysis
- Computing the neighborhood graph
- Embedding the neighborhood graph
- Clustering the neighborhood graph
- Finding marker genes

```
from __future__ import annotations

import matplotlib.pyplot as plt
import pandas as pd

import scanpy as sc
```

In May 2017, this started out as a demonstration that Scanpy would allow to reproduce most of Seurat's [guided clustering tutorial](#) [Satija *et al.*, 2015].

We gratefully acknowledge Seurat's authors for the tutorial! In the meanwhile, we have added and removed a few pieces.

The data consist of *3k PBMCs from a Healthy Donor* and are freely available from 10x Genomics ([here](#) from this [webpage](#)). On a unix system, you can uncomment and run the following to download and unpack the data. The last line creates a directory for writing processed data.

 [latest](#) ▼

```
%%bash
mkdir -p data write
cd data
test -f pbmc3k_filtered_gene_bc_matrices.tar.gz || curl https://cf.10xgenomi
tar -xzf pbmc3k_filtered_gene_bc_matrices.tar.gz
```

### Note

Download the notebook by clicking on the *Edit on GitHub* button. On GitHub, you can download using the *Raw* button via right-click and *Save Link As*. Alternatively, download the whole [scanpy-tutorial](#) repository.

### Note

In Jupyter notebooks and lab, you can see the documentation for a python function by hitting **SHIFT + TAB**. Hit it twice to expand the view.

```
sc.settings.verbosity = 3 # verbosity: errors (0), warnings (1), info (2),
sc.set_figure_params(dpi=80, facecolor="white")
sc.logging.print_header()
```

```
results_file = "write/pbmc3k.h5ad" # the file that will store the analysis
```

Read in the count matrix into an [AnnData](#) object, which holds many slots for annotations and different representations of the data. It also comes with its own HDF5-based file format: **.h5ad**.

```
adata = sc.read_10x_mtx(
    "data/filtered_gene_bc_matrices/hg19/", # the directory with the `.mtx`
    var_names="gene_symbols", # use gene symbols for the variable names (va
    cache=True, # write a cache file for faster subsequent reading
)
```

... reading from cache file cache/data-filtered\_gene\_bc\_ma

 latest ▼

```
adata
```

```
AnnData object with n_obs × n_vars = 2700 × 32738  
var: 'gene_ids'
```

#### Note

See [Getting started with anndata](#) for a more comprehensive introduction to `AnnData`.

```
adata.var_names_make_unique() # this is unnecessary if using `var_names='ge
```

```
# To match R results, do: `adata.X = adata.X.astype("int32")`  
adata
```

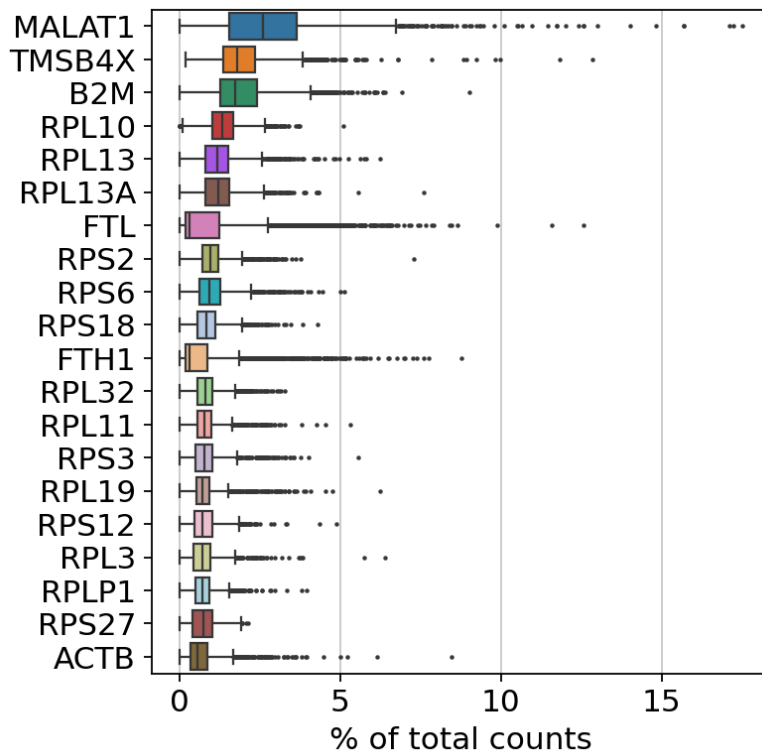
```
AnnData object with n_obs × n_vars = 2700 × 32738  
var: 'gene_ids'
```

## Preprocessing

Show those genes that yield the highest fraction of counts in each single cell, across all cells.

```
sc.pl.highest_expr_genes(adata, n_top=20)
```

```
normalizing counts per cell  
finished (0:00:01)
```



Basic filtering:

```
sc.pp.filter_cells(adata, min_genes=200) # this does nothing, in this speci
sc.pp.filter_genes(adata, min_cells=3)
adata
```

filtered out 19024 genes that are detected in less than 3 cells

```
AnnData object with n_obs × n_vars = 2700 × 13714
  obs: 'n_genes'
  var: 'gene_ids', 'n_cells'
```

Let's assemble some information about mitochondrial genes, which are important for quality control.

Citing from "Simple Single Cell" workflows [[Lun et al., 2016](#)]:

High proportions are indicative of poor-quality cells [[Illicic et al., 2016](#), [Islam et al., 2013](#)], possibly because of loss of cytoplasmic RNA from perforated cells. The reasoning is that mitochondria are larger than individual transcript molecules and less likely to escape through tears in the cell membrane.

[latest](#) ▼

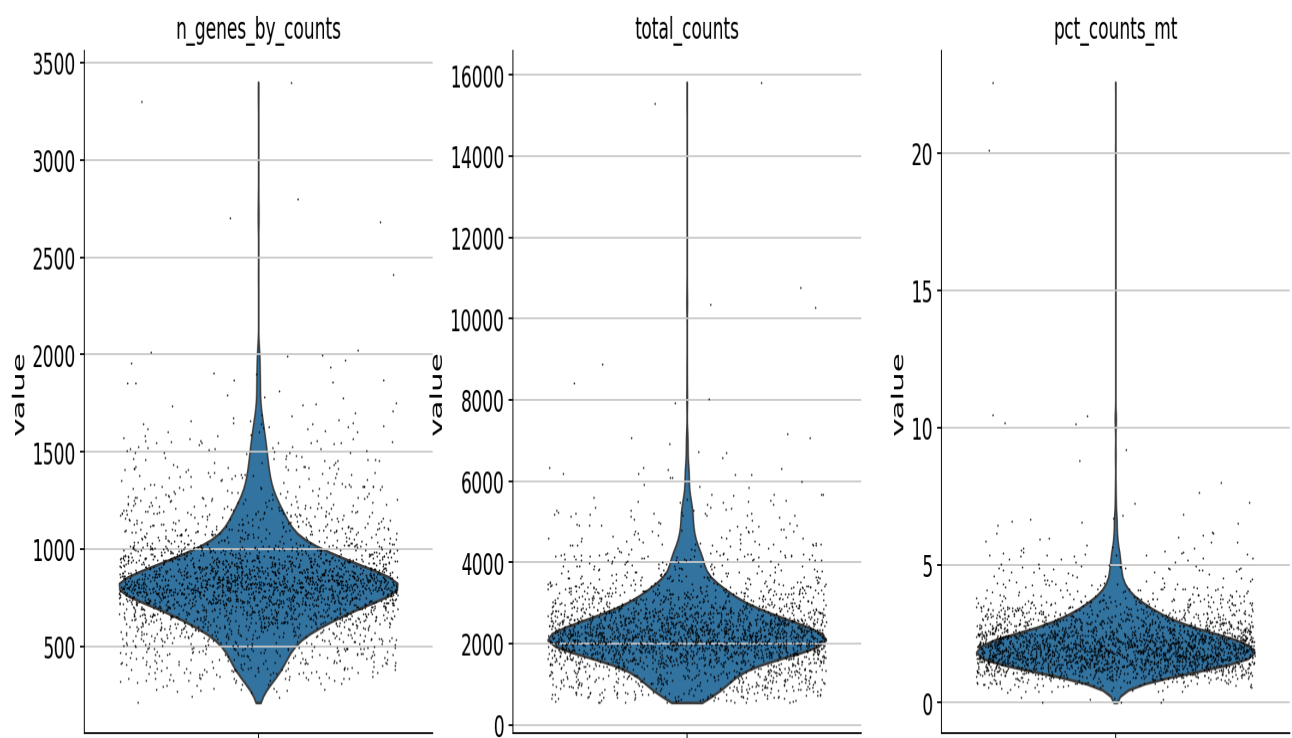
With `pp.calculate_qc_metrics`, we can compute many metrics very efficiently.

```
# annotate the group of mitochondrial genes as "mt"
adata.var["mt"] = adata.var_names.str.startswith("MT-")
sc.pp.calculate_qc_metrics(
    adata, qc_vars=["mt"], percent_top=None, log1p=False, inplace=True
)
```

A violin plot of some of the computed quality measures:

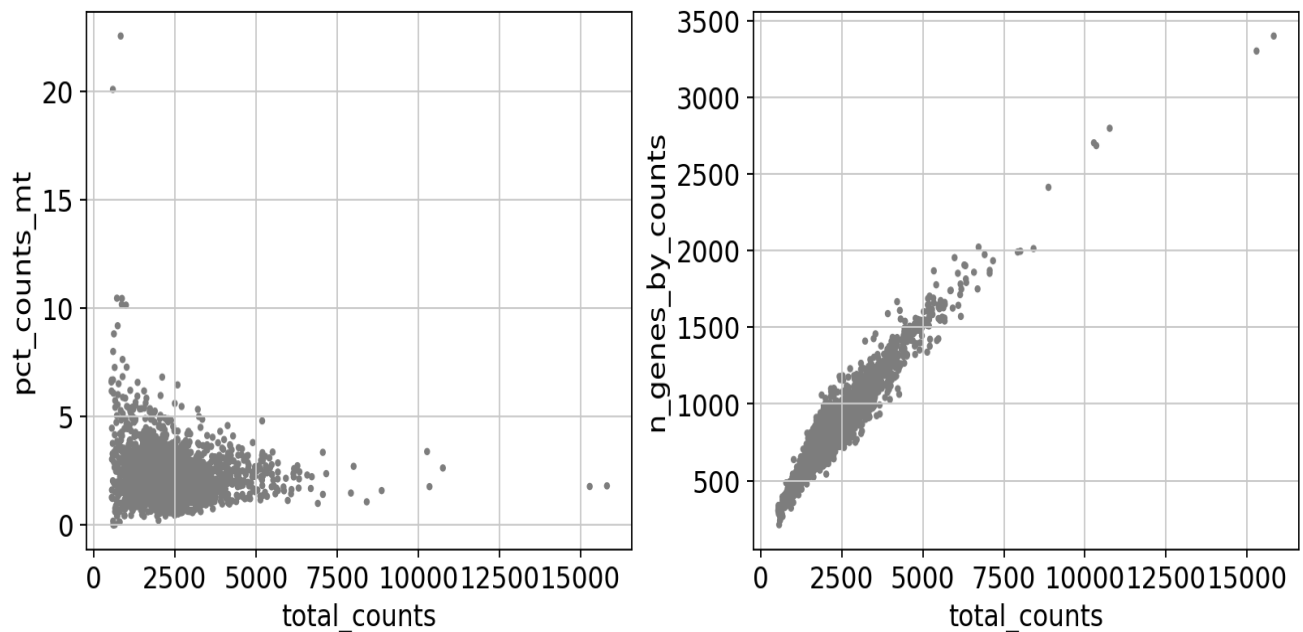
- the number of genes expressed in the count matrix
- the total counts per cell
- the percentage of counts in mitochondrial genes

```
sc.pl.violin(
    adata,
    ["n_genes_by_counts", "total_counts", "pct_counts_mt"],
    jitter=0.4,
    multi_panel=True,
)
```



Remove cells that have too many mitochondrial genes expressed or too many total counts:

```
fig, axs = plt.subplots(1, 2, figsize=(10, 4), layout="constrained")
sc.pl.scatter(adata, x="total_counts", y="pct_counts_mt", show=False, ax=axs[0])
sc.pl.scatter(adata, x="total_counts", y="n_genes_by_counts", show=False, ax=axs[1])
```



Actually do the filtering by slicing the `AnnData` object.

```
adata = adata[
    (adata.obs.n_genes_by_counts < 2500)
    & (adata.obs.n_genes_by_counts > 200)
    & (adata.obs.pct_counts_mt < 5),
    :,
].copy()
adata.layers["counts"] = adata.X.copy()
```

adata

```
AnnData object with n_obs × n_vars = 2638 × 13714
  obs: 'n_genes', 'n_genes_by_counts', 'total_counts', 'total_counts_mt', 'pct_counts_mt'
  var: 'gene_ids', 'n_cells', 'mt', 'n_cells_by_counts', 'mean_counts', 'pct_counts_mt'
  layers: 'counts'
```

Total-count normalize (library-size correct) the data matrix **X** to 10,000 reads per cell, so that counts become comparable among cells.

```
sc.pp.normalize_total(adata, target_sum=1e4)
```

```
normalizing counts per cell
finished (0:00:00)
```

 [latest](#) ▼

Logarithmize the data:

```
sc.pp.log1p(adata)
```

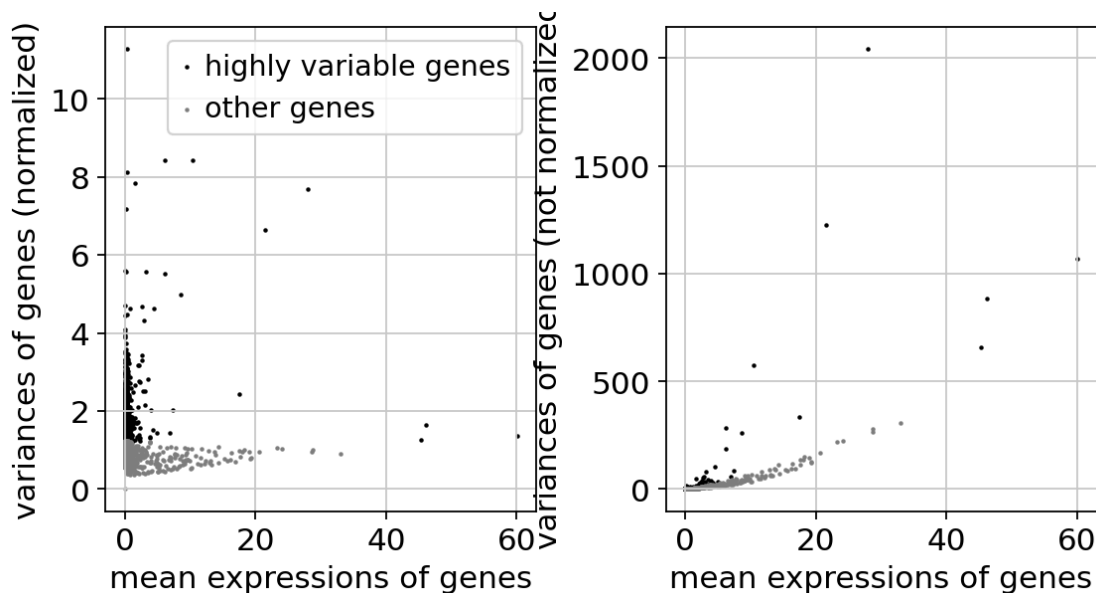
Identify highly-variable genes. Up to ties, this should match exactly the tutorial from Seurat: [https://satijalab.org/seurat/articles/pbmc3k\\_tutorial](https://satijalab.org/seurat/articles/pbmc3k_tutorial)

```
sc.pp.highly_variable_genes(
    adata,
    layer="counts",
    n_top_genes=2000,
    min_mean=0.0125,
    max_mean=3,
    min_disp=0.5,
    flavor="seurat_v3",
)
```

extracting highly variable genes

```
--> added
    'highly_variable', boolean vector (adata.var)
    'highly_variable_rank', float vector (adata.var)
    'means', float vector (adata.var)
    'variances', float vector (adata.var)
    'variances_norm', float vector (adata.var)
```

```
sc.pl.highly_variable_genes(adata)
```



**Note**

The result of the previous highly-variable-genes detection is stored as an annotation in `.var["highly_variable"]` and auto-detected by PCA and hence, `sc.pp.neighbors` and subsequent manifold/graph tools. In that case, the step *actually do the filtering* below is unnecessary, too.

Scale each gene to unit variance. Clip values exceeding standard deviation 10. To match [Seurat's pbmc 3k tutorial](#), you can elect to regress out only `pct_counts_mt` and subset (which appears to be optional in their tutorial).

```
adata.layers["scaled"] = adata.X.toarray()
sc.pp.regress_out(adata, ["total_counts", "pct_counts_mt"], layer="scaled")
sc.pp.scale(adata, max_value=10, layer="scaled")
```

```
regressing out ['total_counts', 'pct_counts_mt']
finished (0:00:02)
```

```
/home/phil/.local/share/hatch/env/virtual/scanpy-tutorials/vsyQHp6L/notebook,
data[i] -= regressor[i] @ coeff
/home/phil/.local/share/hatch/env/virtual/scanpy-tutorials/vsyQHp6L/notebook,
data[i] -= regressor[i] @ coeff
```

## Principal component analysis

Reduce the dimensionality of the data by running principal component analysis (PCA), which reveals the main axes of variation and denoises the data.

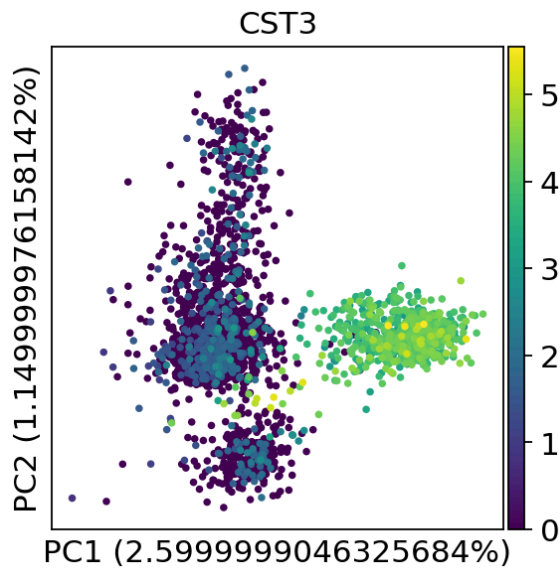
```
sc.pp.pca(adata, layer="scaled", svd_solver="arpack")
```

```
computing PCA
with n_comps=50
finished (0:00:01)
```

We can make a scatter plot in the PCA coordinates, but we will not use

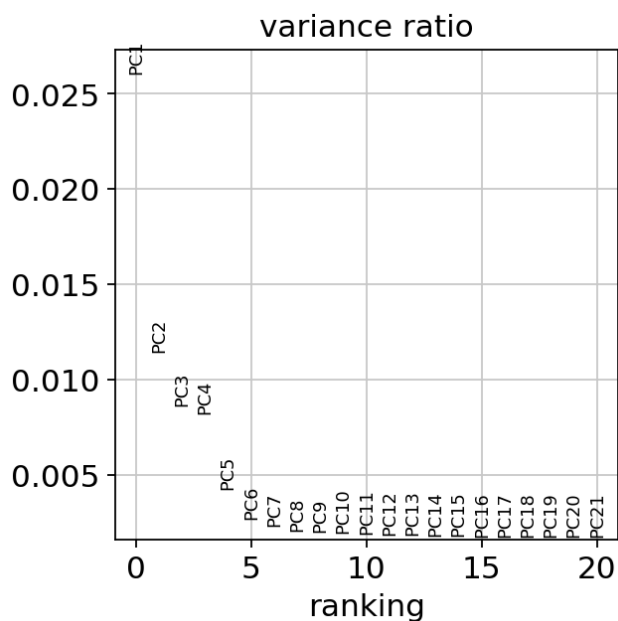
 [latest](#) ▼

```
sc.pl.pca(adata, annotate_var_explained=True, color="CST3")
```



Let us inspect the contribution of single PCs to the total variance in the data. This gives us information about how many PCs we should consider in order to compute the neighborhood relations of cells, e.g. used in the clustering function `sc.tl.louvain()` or tSNE `sc.tl.tsne()`. In our experience, often a rough estimate of the number of PCs does fine.

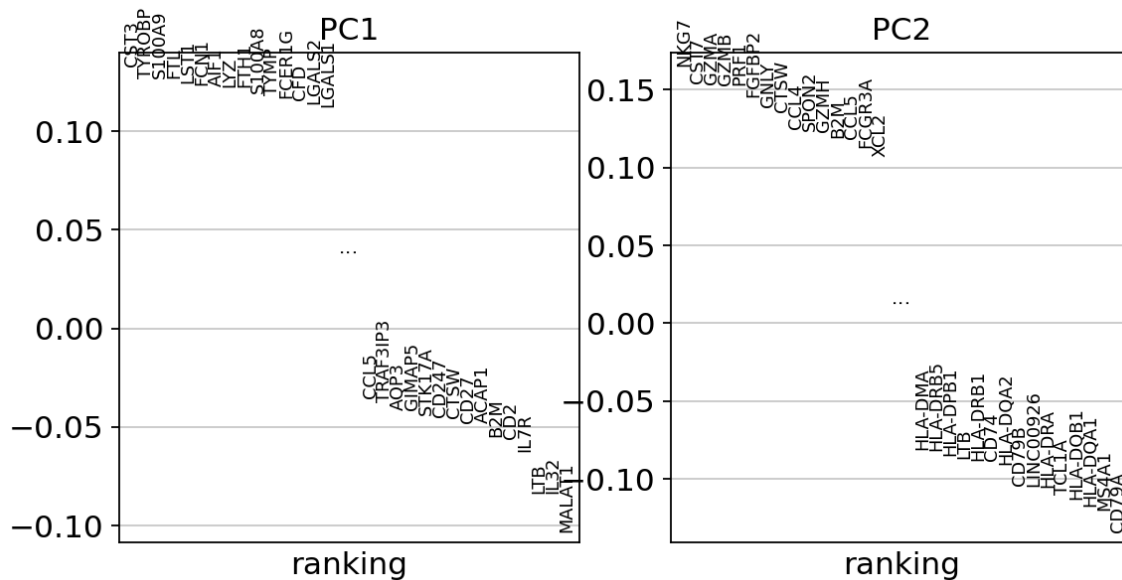
```
sc.pl.pca_variance_ratio(adata, n_pcs=20)
```



While our algorithms differ, yielding slightly different components, the rankings match those found in [the Seurat tutorial](https://seurat.tutorial) modulo the sign.

 [latest](#) ▼

```
sc.pl.pca_loadings(adata, components=(1, 2), include_lowest=True)
```



## Computing the neighborhood graph

Let us compute the neighborhood graph of cells using the PCA representation of the data matrix. You might simply use default values here. For the sake of reproducing Seurat's results, let's take the following values.

```
sc.pp.neighbors(adata, n_neighbors=10, n_pcs=40)
```

```
computing neighbors
  using 'X_pca' with n_pcs = 40
  finished: added to `uns['neighbors']`
  `.obs['distances']`, distances for each pair of neighbors
  `.obs['connectivities']`, weighted adjacency matrix (0:00:04)
```

## Embedding the neighborhood graph

We suggest embedding the graph in two dimensions using UMAP [McInnes et al., 2018], see below. It is potentially more faithful to the global connectivity of the manifold than tSNE, i.e., it better preserves trajectories. In some occasions, you might still observe disconnected clusters and similar connectivity violations. They can usually be remedied by running:

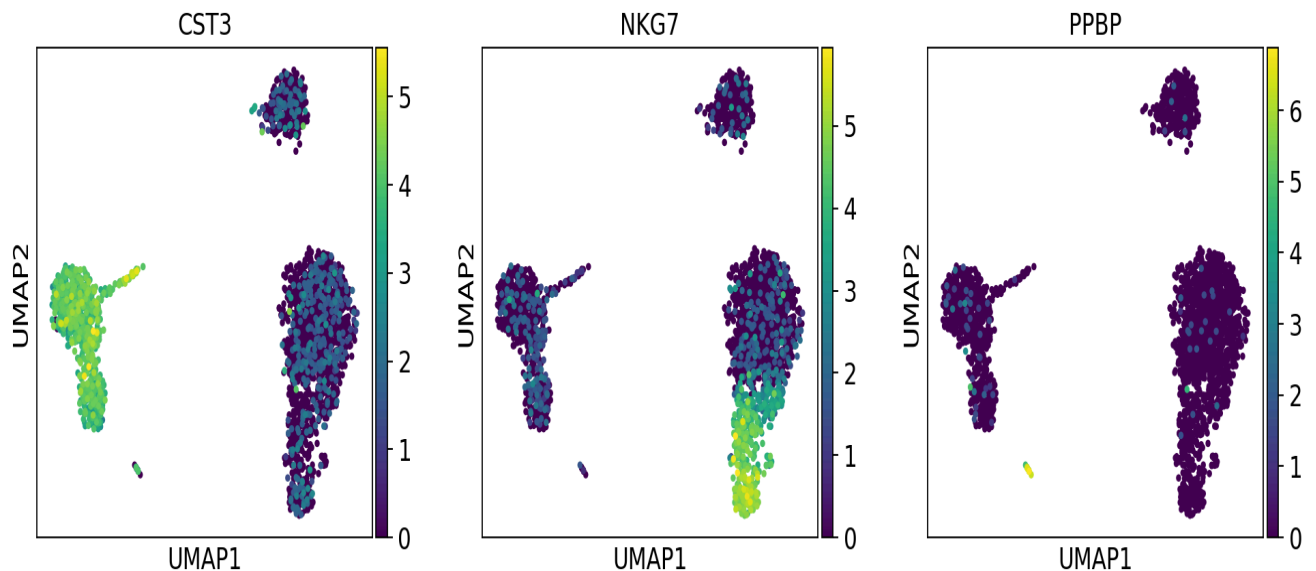
 [latest](#) ▼

```
sc.tl.paga(adata)
sc.pl.paga(adata, plot=False) # remove `plot=False` if you want to see the
sc.tl.umap(adata, init_pos='paga')
```

```
sc.tl.umap(adata)
```

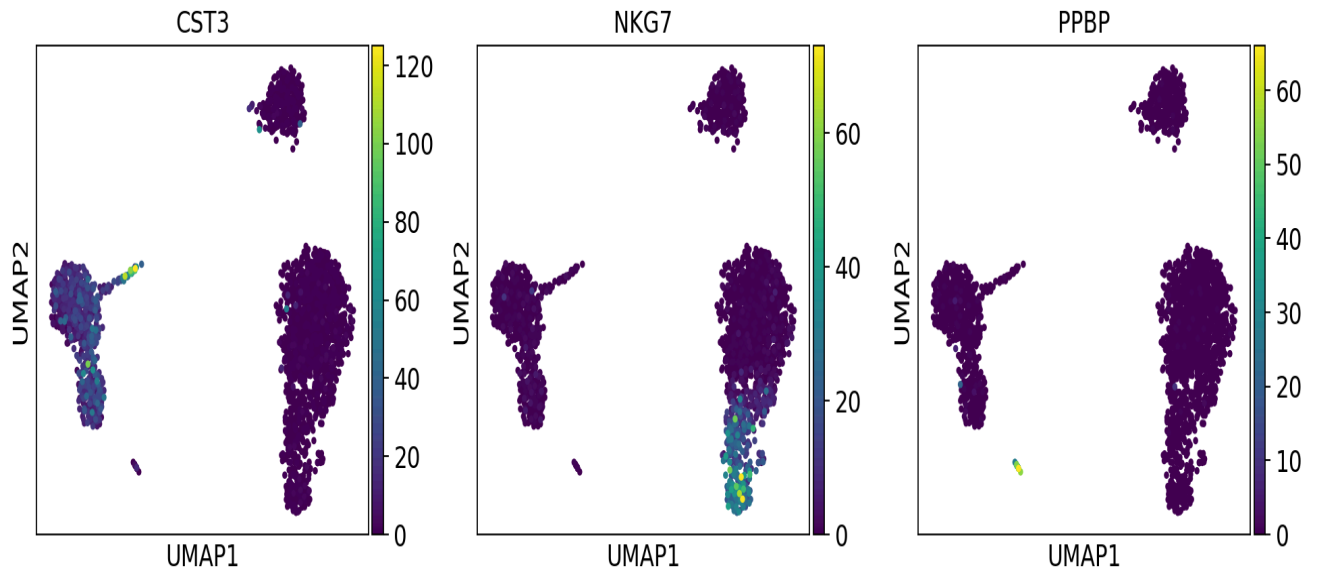
```
computing UMAP
finished: added
'X_umap', UMAP coordinates (adata.obsm)
'umap', UMAP parameters (adata.uns) (0:00:02)
```

```
sc.pl.umap(adata, color=["CST3", "NKG7", "PPBP"])
```

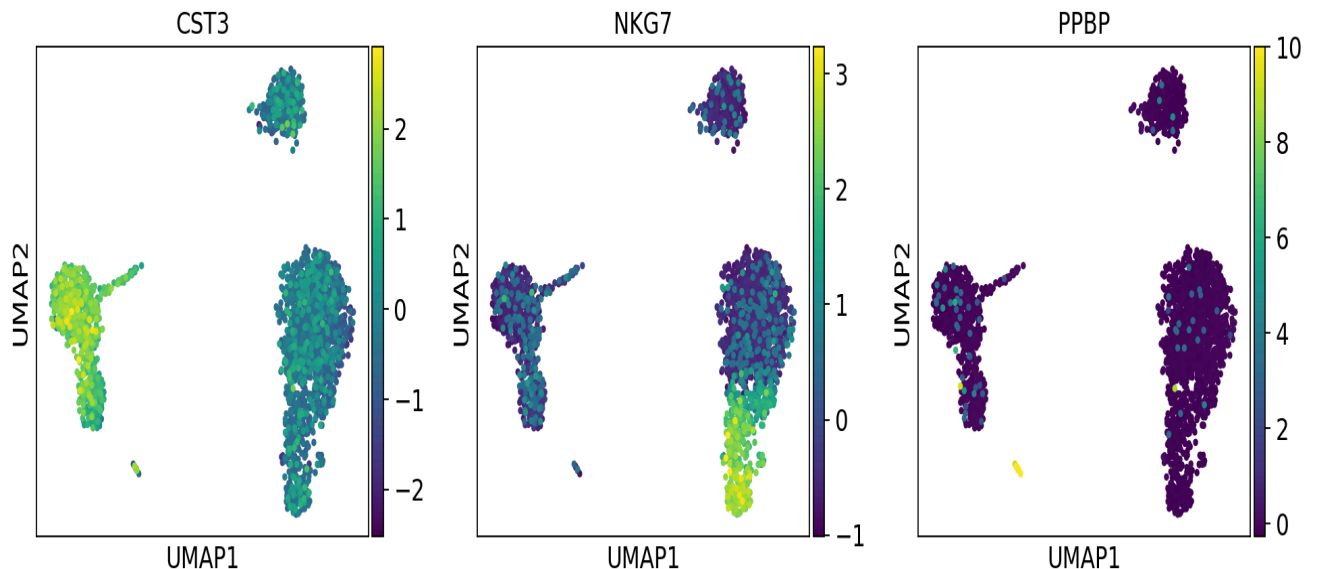


As we set the `.X` attribute of `adata` to be the normalized data, the previous plots showed the normalized gene expression. You can also plot the counts directly, or the “scaled\_hvg” (normalized, logarithmized, and scaled) data.

```
sc.pl.umap(adata, color=["CST3", "NKG7", "PPBP"], layer="counts")
```



```
sc.pl.umap(adata, color=["CST3", "NKG7", "PPBP"], layer="scaled")
```



## Clustering the neighborhood graph

As with Seurat and many other frameworks, we recommend the Leiden graph-clustering method (community detection based on optimizing modularity) by Traag *et al.* [2019]. Note that Leiden clustering directly clusters the neighborhood graph of cells, which we already computed in the previous section.

```

sc.tl.leiden(
    adata,
    resolution=0.7,
    random_state=0,
    flavor="igraph",
    n_iterations=2,
    directed=False,
)
adata.obs["leiden"] = adata.obs["leiden"].copy()
adata.uns["leiden"] = adata.uns["leiden"].copy()
adata.obsm["X_umap"] = adata.obsm["X_umap"].copy()

```

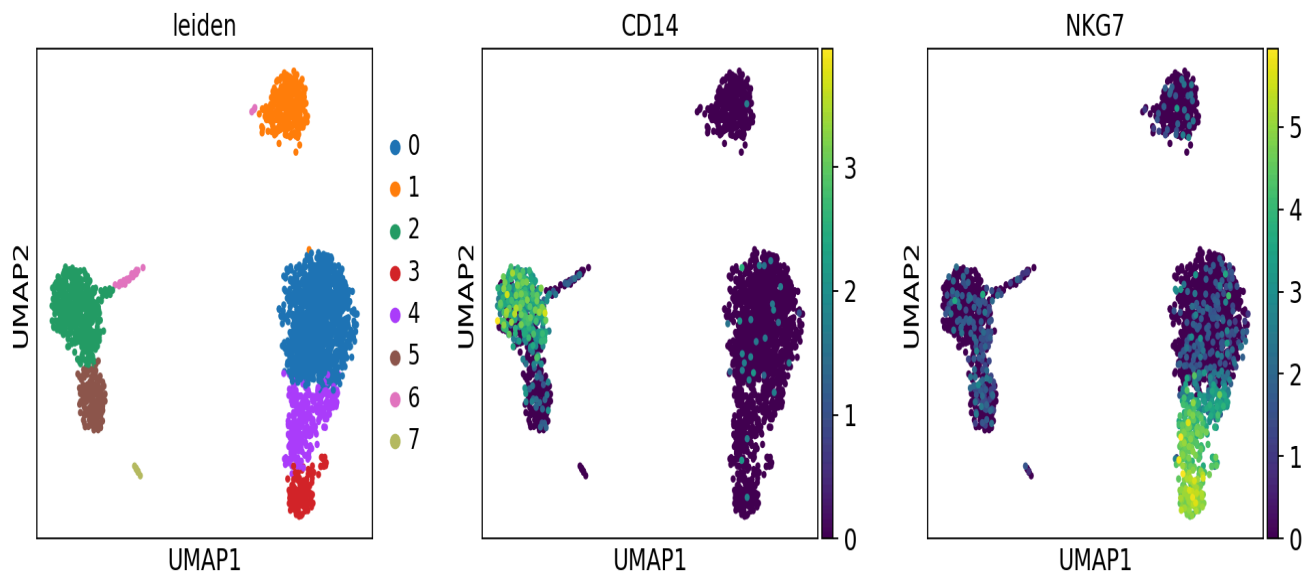
```

running Leiden clustering
finished: found 8 clusters and added
'leiden', the cluster labels (adata.obs, categorical) (0:00:00)

```

Plot the clusters, which agree quite well with the result of Seurat.

```
sc.pl.umap(adata, color=["leiden", "CD14", "NKG7"])
```



## Finding marker genes

Let us compute a ranking for the highly differential genes in each cluster. The simplest and fastest method to do so is the t-test.

```

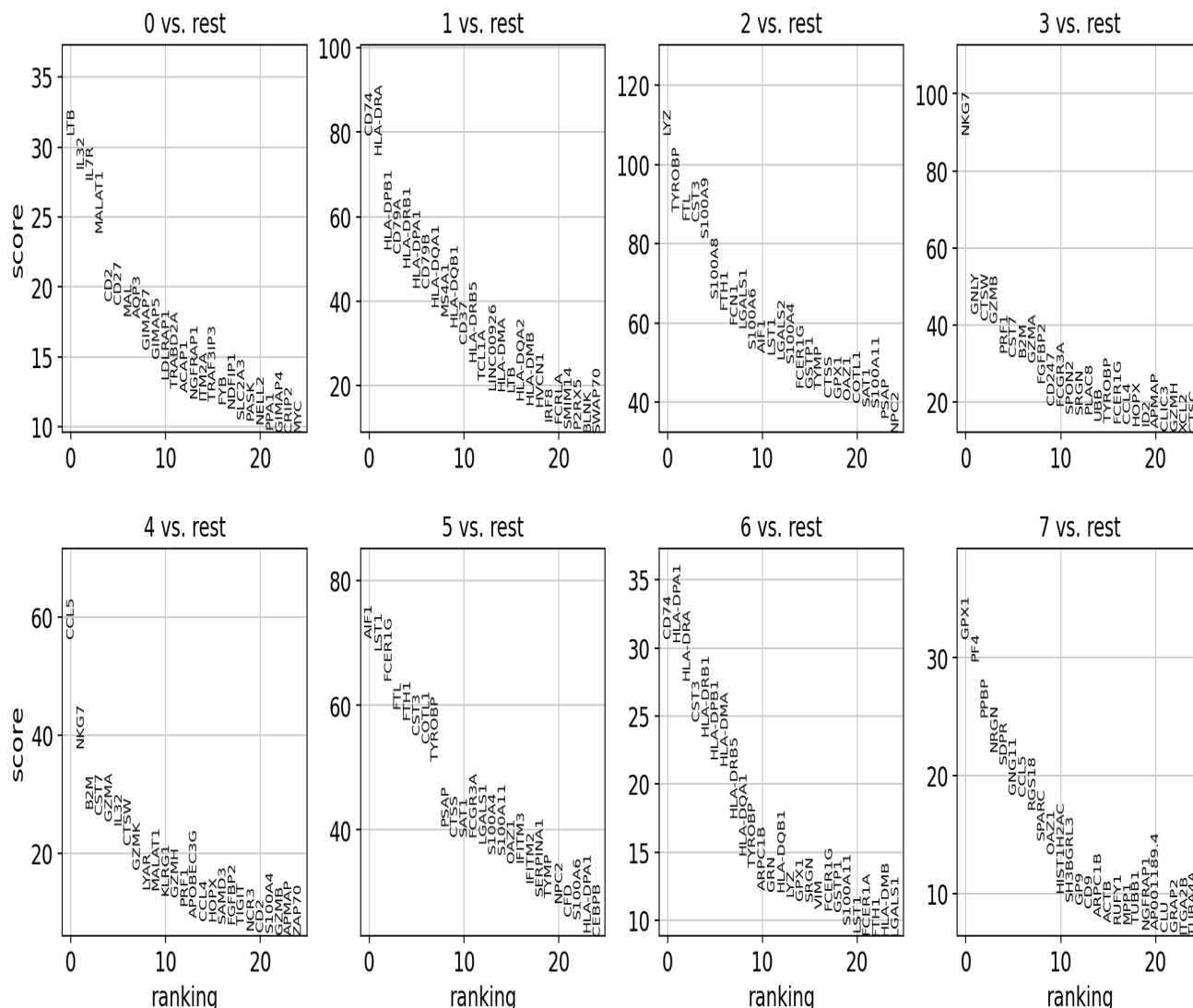
sc.tl.rank_genes_groups(adata, "leiden", mask="var.highly_variable", method=
sc.pl.rank_genes_groups(adata, n_genes=25, sharey=False)

```

[latest](#) ▼

## ranking genes

```
finished: added to `uns['rank_genes_groups']`
'names', sorted np.recarray to be indexed by group ids
'scores', sorted np.recarray to be indexed by group ids
'logfoldchanges', sorted np.recarray to be indexed by group ids
'pvals', sorted np.recarray to be indexed by group ids
'pvals_adj', sorted np.recarray to be indexed by group ids (0:00:00)
```



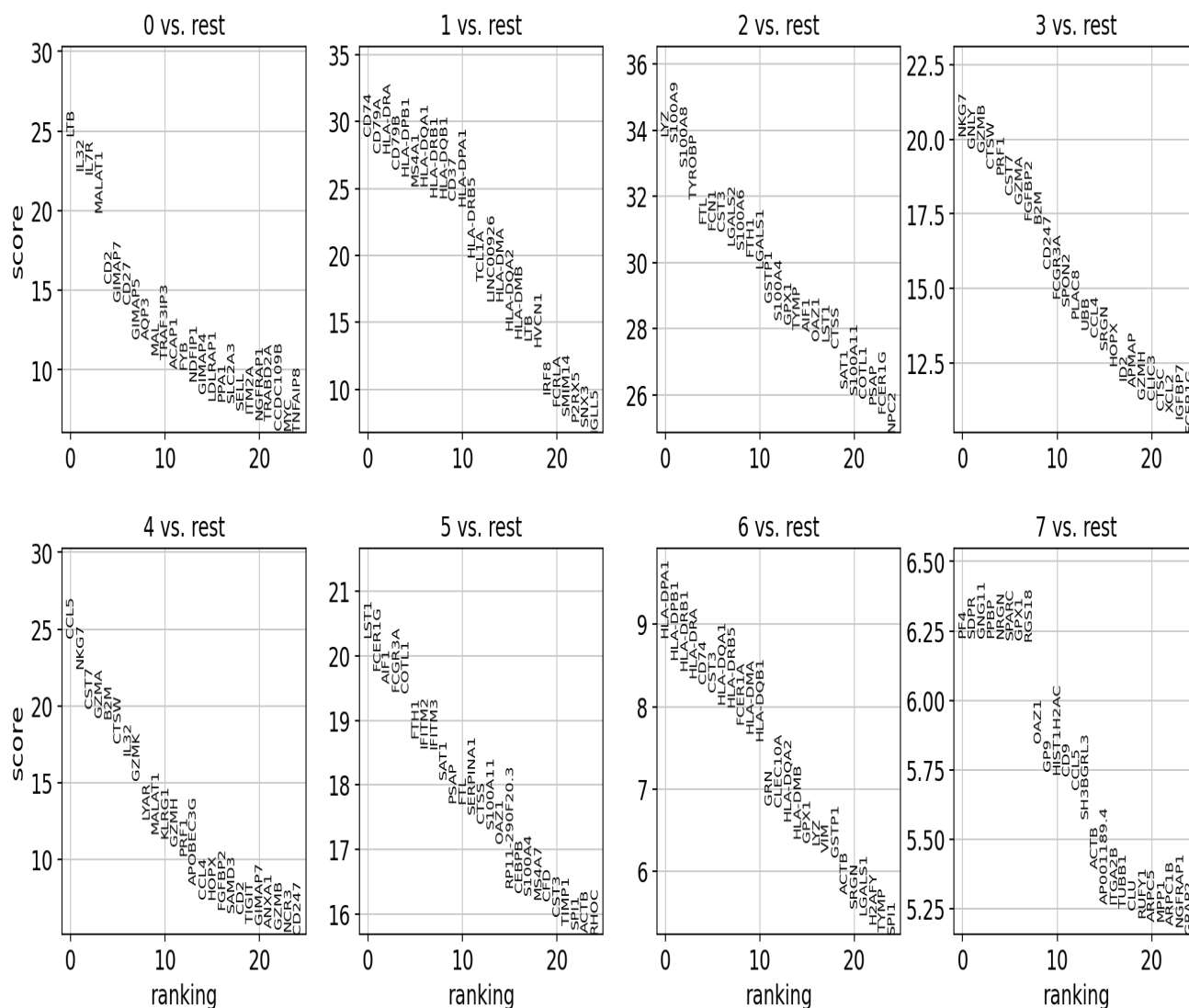
```
sc.settings.verbosity = 2 # reduce the verbosity
```

The result of a [Wilcoxon rank-sum \(Mann-Whitney-U\)](#) test is very similar. We recommend using the latter in publications, see e.g., Sonesson and Robinson [2018]. You might also consider much more powerful differential testing packages like MAST, limma, DESeq2 and, for python, the recent diffxpy.

```
sc.tl.rank_genes_groups(adata, "leiden", mask="var.highly_variable", method=
sc.pl.rank_genes_groups(adata, n_genes=25, sharey=False)
```

[latest](#) ▼

```
ranking genes
  finished (0:00:03)
```



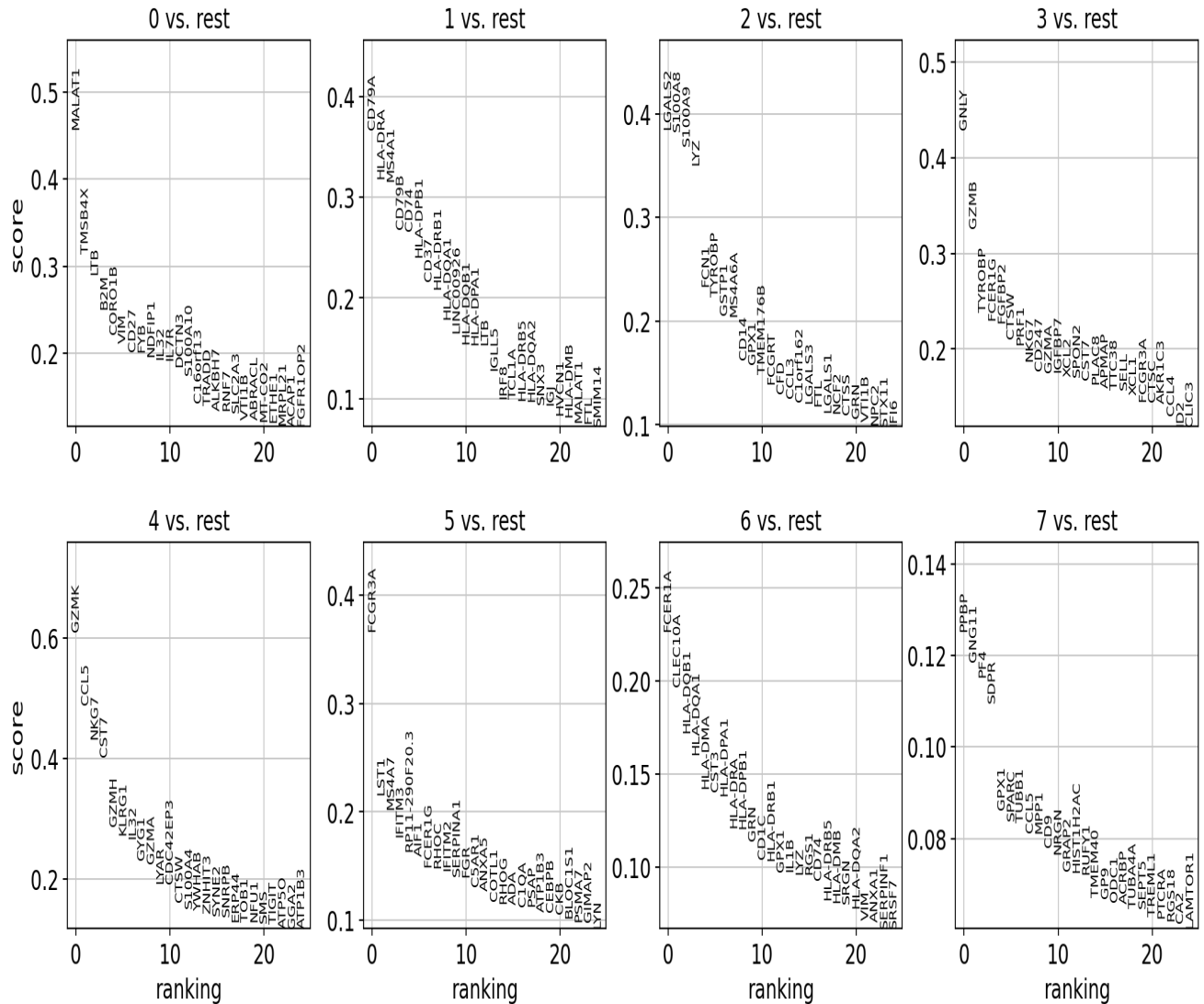
Save the result.

```
adata.write(results_file)
```

As an alternative, let us rank genes using logistic regression. For instance, this has been suggested by Ntranos *et al.* [2019]. The essential difference is that here, we use a multi-variate approach whereas conventional differential tests are uni-variate. Clark *et al.* [2014] has more details.

```
sc.tl.rank_genes_groups(
    adata, "leiden", mask="var.highly_variable", method="logreg", max_iter=1
)
sc.pl.rank_genes_groups(adata, n_genes=25, sharey=False)
```

ranking genes  
finished (0:00:03)



Most of these genes are found in the highly expressed genes with notable exceptions like **CD8A** (see the below dotplot).

| Leiden Group | Markers       | Cell Type         |
|--------------|---------------|-------------------|
| 0            | IL7R          | CD4 T cells       |
| 1            | CD14, LYZ     | CD14+ Monocytes   |
| 2            | MS4A1         | B cells           |
| 3            | CD8A          | CD8 T cells       |
| 4            | GNLY, NKG7    | NK cells          |
| 5            | FCGR3A, MS4A7 | FCGR3A+ Monocytes |
| 6            | FCER1A, CST3  | Dendritic Cells   |
| 7            | PPBP          | Megakaryocytes    |

Let us also define a list of marker genes for later reference.

```
marker_genes = [
    *["IL7R", "CD79A", "MS4A1", "CD8A", "CD8B", "LYZ", "CD14"],
    *["LGALS3", "S100A8", "GNLY", "NKG7", "KLRB1"],
    *["FCGR3A", "MS4A7", "FCER1A", "CST3", "PPBP"],
]
```

```
adata = sc.read(results_file)
```

Show the 10 top ranked genes per cluster 0, 1, ..., 7 in a dataframe.

```
pd.DataFrame(adata.uns["rank_genes_groups"]["names"]).head(5)
```

|   | 0      | 1        | 2      | 3    | 4    | 5      | 6        | 7     |
|---|--------|----------|--------|------|------|--------|----------|-------|
| 0 | LTB    | CD74     | LYZ    | NKG7 | CCL5 | LST1   | HLA-DPA1 | PF4   |
| 1 | IL32   | CD79A    | S100A9 | GNLY | NKG7 | FCER1G | HLA-DPB1 | SDPR  |
| 2 | IL7R   | HLA-DRA  | S100A8 | GZMB | CST7 | AIF1   | HLA-DRB1 | GNG11 |
| 3 | MALAT1 | CD79B    | TYROBP | CTSW | GZMA | FCGR3A | HLA-DRA  | PPBP  |
| 4 | CD2    | HLA-DPB1 | FTL    | PRF1 | B2M  | COTL1  | CD74     | NRGN  |

Get a table with the scores and groups.

 [latest](#) ▼

```

result = adata.uns["rank_genes_groups"]
groups = result["names"].dtype.names
pd.DataFrame({
    f"{group}_{key[:1]}": result[key][group]
    for group in groups
    for key in ["names", "pvals"]
}).head(5)

```

|   | 0_n    | 0_p           | 1_n      | 1_p           | 2_n    | 2_p           | 3_n  | 3_p          | 4_n  |
|---|--------|---------------|----------|---------------|--------|---------------|------|--------------|------|
| 0 | LTB    | 4.523933e-135 | CD74     | 1.972305e-184 | LYZ    | 5.016654e-251 | NKG7 | 4.220384e-90 | CCL5 |
| 1 | IL32   | 1.391499e-112 | CD79A    | 1.882747e-169 | S100A9 | 1.126831e-248 | GNLY | 9.274173e-87 | NKG7 |
| 2 | IL7R   | 2.756677e-110 | HLA-DRA  | 2.840744e-168 | S100A8 | 6.457752e-238 | GZMB | 1.164234e-85 | CST7 |
| 3 | MALAT1 | 4.433395e-88  | CD79B    | 1.868216e-154 | TYROBP | 1.259221e-223 | CTSW | 5.115917e-81 | GZMA |
| 4 | CD2    | 4.504927e-54  | HLA-DPB1 | 3.333386e-148 | FTL    | 1.887168e-213 | PRF1 | 3.618719e-79 | B2M  |

Compare to a single cluster:

```

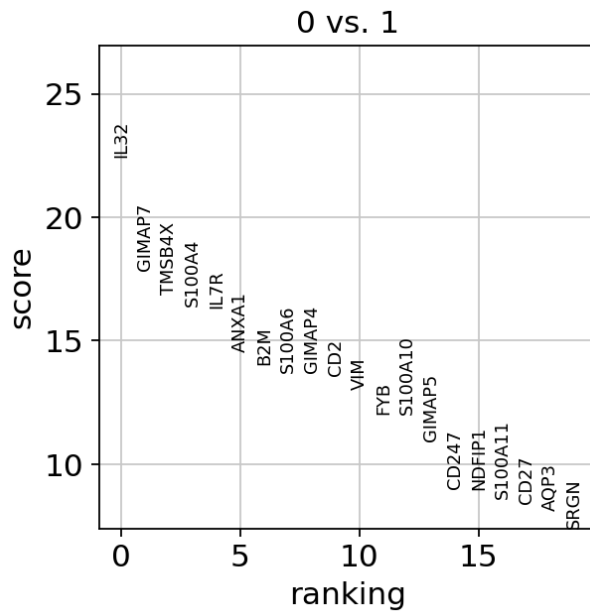
sc.tl.rank_genes_groups(
    adata,
    "leiden",
    mask="var.highly_variable",
    groups=["0"],
    reference="1",
    method="wilcoxon",
)
sc.pl.rank_genes_groups(adata, groups=["0"], n_genes=20)

```

```

ranking genes
finished (0:00:00)

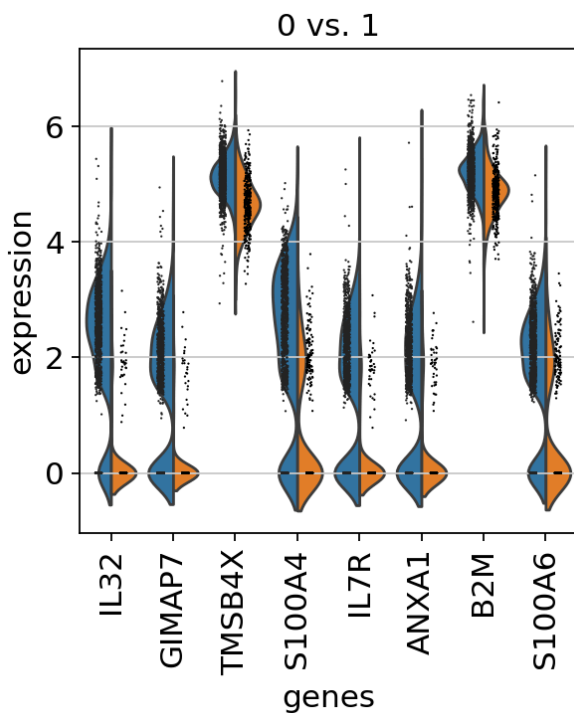
```



If we want a more detailed view for a certain group, use

`sc.pl.rank_genes_groups_violin`.

```
sc.pl.rank_genes_groups_violin(adata, groups="0", n_genes=8)
```

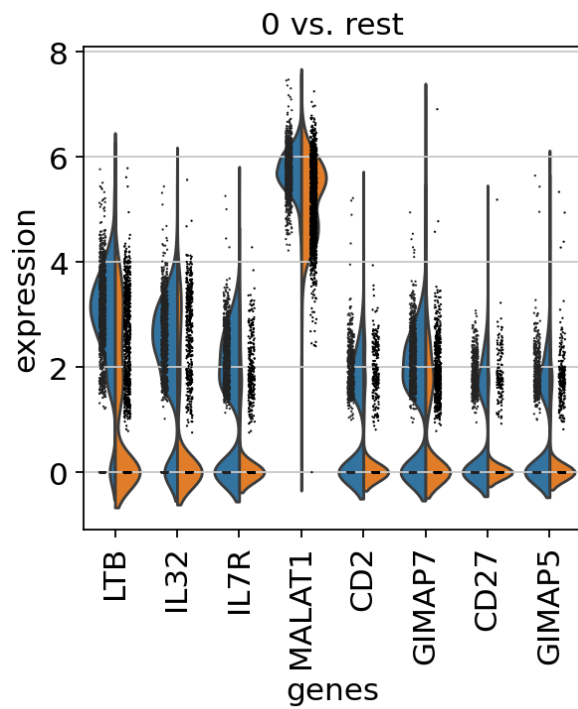


Reload the object with the computed differential expression (i.e. DE via a comparison with the rest of the groups):

```
adata = sc.read(results_file)
```

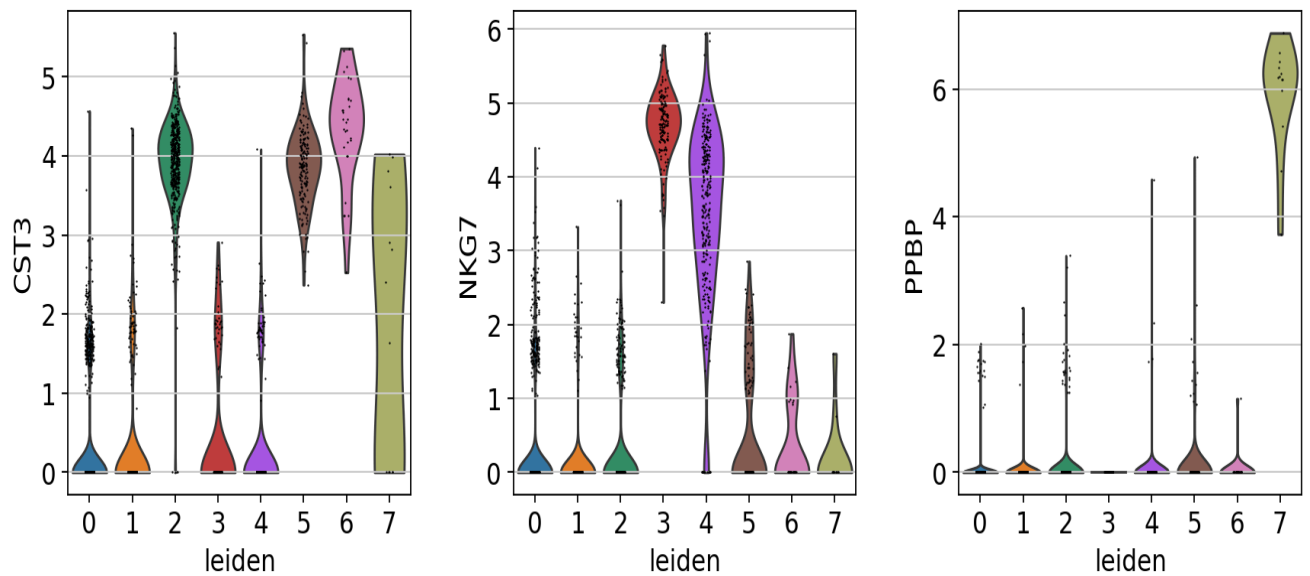
```
sc.pl.rank_genes_groups_violin(adata, groups="0", n_genes=8)
```

[latest](#) ▼



If you want to compare a certain gene across groups, use the following.

```
sc.pl.violin(adata, ["CST3", "NKG7", "PPBP"], groupby="leiden")
```

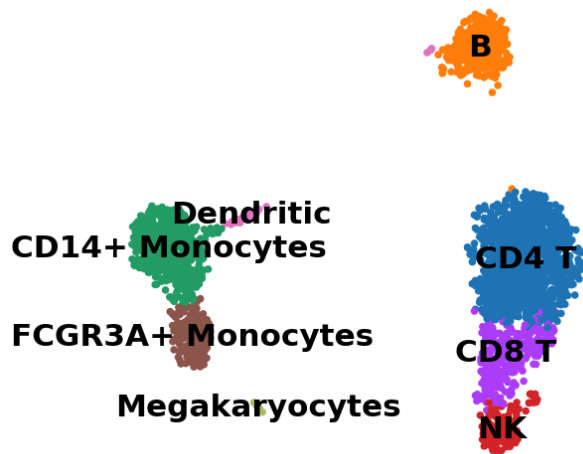


Actually mark the cell types.

```
new_cluster_names = [
    "CD4 T",
    "B",
    "CD14+ Monocytes",
    "NK",
    "CD8 T",
    "FCGR3A+ Monocytes",
    "Dendritic",
    "Megakaryocytes",
]
adata.rename_categories("leiden", new_cluster_names)
```

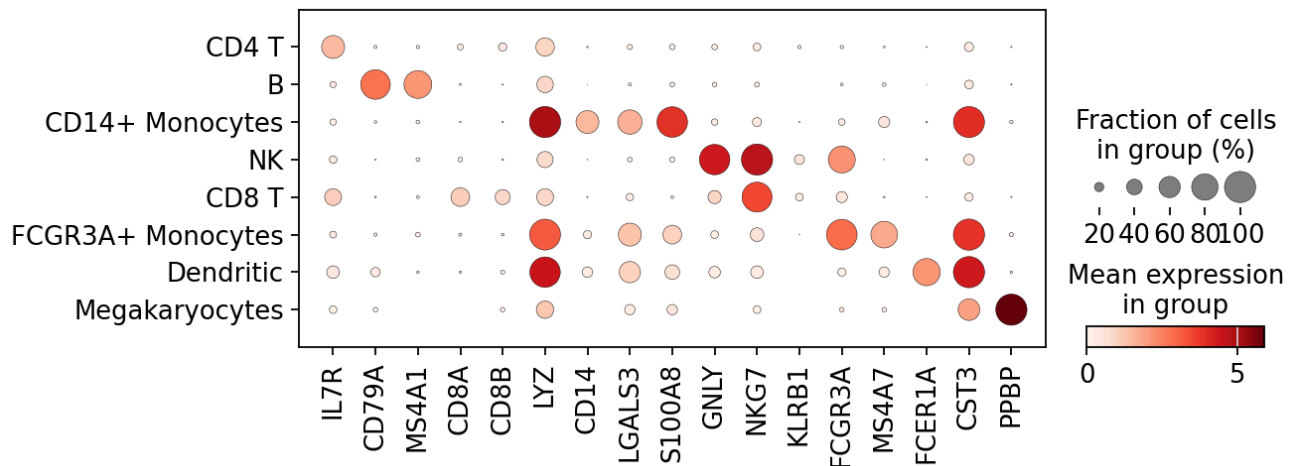
```
sc.pl.umap(adata, color="leiden", legend_loc="on data", title="", frameon=False)
```

WARNING: saving figure to file figures/umap.pdf



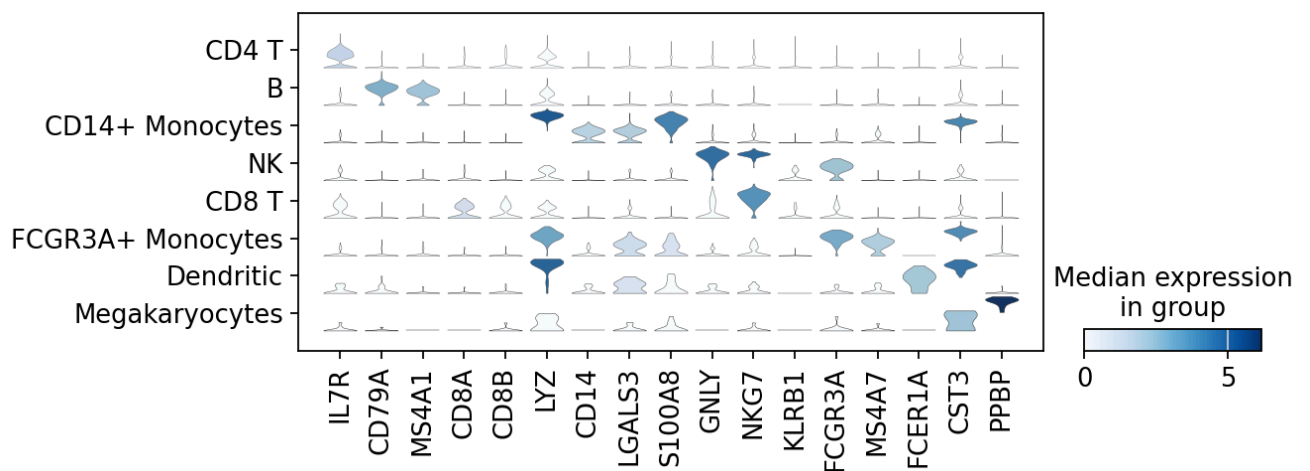
Now that we annotated the cell types, let us visualize the marker genes.

```
sc.pl.dotplot(adata, marker_genes, groupby="leiden")
```



There is also a very compact violin plot.

```
sc.pl.stacked_violin(adata, marker_genes, groupby="leiden")
```



During the course of this analysis, the AnnData accumulated the following annotations.

```
adata
```

```
AnnData object with n_obs × n_vars = 2638 × 13714
  obs: 'n_genes', 'n_genes_by_counts', 'total_counts', 'total_counts_mt', '
  var: 'gene_ids', 'n_cells', 'mt', 'n_cells_by_counts', 'mean_counts', 'pc
  uns: 'hvg', 'leiden', 'leiden_colors', 'log1p', 'neighbors', 'pca', 'rank
  obsm: 'X_pca', 'X_umap'
  varm: 'PCs'
  layers: 'counts', 'scaled'
  obsp: 'connectivities', 'distances'
```

```
# `compression='gzip'` saves disk space, and slightly slows down writing and
adata.write(results_file, compression="gzip")
```

Get a rough overview of the file using `h5ls`, which has many options - for more details see [here](#). The file format might still be subject to further optimization in the future. All reading functions will remain backwards-compatible, though.